Utilisation du programme de simulation d'une machine virtuelle

Ce guide explique comment compiler le projet, rédiger un script assembleur valide, et exécuter le simulateur via la ligne de commande.

I. Prérequis et compilation

1. Préparer les fichiers

Assurez-vous que les fichiers : main.c, assembleur.c, assembleur.h, codage.c, codage.h, commun.h, cpu.c et cpu.h soient bien téléchargés correctement dans un même répertoire connu.

2. Compiler le programme

Ouvrez un terminal de commande et placer vous dans le répertoire où se situent les fichiers composant le programme énuméré si contre.

Lancez la commande suivante :

Sur MacOs/Linux : gcc *.c -Wall -o simulateur

Sur Windows : gcc *.c -Wall -o simulateur.exe

Si la compilation réussie, aucun message d'erreur ne doit apparaitre et un fichier exécutable nommé simulateur ou simulateur.exe est créé.

Vous avez simplement à taper ceci pour lancer l'exécution de votre programme :

- Sur MacOs/Linux : ./simulateur.out nomdufichier.txt
- Sur Windows : simulateur nomdufichier.txt

II. Rédaction d'un programme assembleur

Les programmes doivent être écrit dans des fichiers .txt ou .asm pour une exécution sans erreur et valide.

Syntaxe générale :

Une ligne d'un programme assembleur suit une syntaxe précise :

[label] : Mnémonique [Destination], [Source1], [Source2] [; Commentaire]

Quelques règles d'écritures :

1. Label : le label est utile pour faire une référence a un point du code auquel le processeur est amené à revenir. Une étiquette sans appel entraine un Warning de la machine.
2. Mnémonique/code opérateur ou opcode en anglais (pour les plus fort…) : Ce sont les mots clés permettant de définir les opérations effectuées par le processeur. Ils sont insensibles à la casse (ADD, Add, add sont valides). La liste des opérations autorisé est donné à la fin du document.
3. Les registres : Ce sont les ‘’variables’’ du processeur, elles vont de R0 à R31, insensible à la casse également (R0 est équivalent à r0). (A noter que le registre r0 vaut toujours 0, si une référence de changement apparait sur ce registre cela entraine un Warning)
4. Valeurs immédiates : les nombres décimaux et hexadécimaux sont tolérés. Cependant il est important de noter que la machine est programmée sur 16 bit, donc les valeurs vont de -32768 à 65535.
5. Accès mémoire : les accès mémoires doivent impérativement contenir des parenthèses. Pour la lecture : LDB R1, (R2). Pour l'écriture : STB (R1), R2
6. Commentaire : Pour les commentaires, le style assembleur et C sont tolérés, c'est-à-dire ceux-là : ‘ ;’, ‘/** **/’, ‘//’.

Voici un exemple de langage assembleur valide :

```
; Programme de test : Somme de deux nombres
debut:          //Warning car pas de référence à début dans la suite du code
  IN R1         //Demande une valeur (ex: 5)
  ADD R2, R0, #10   ; R2 = 10
  ADD R3, R1, R2    ; R3 = R1 + 1
  OUT R3        ; Affiche le résultat
  HLT           ; Arrêt obligatoire sinon erreur
```

III. Exécution du programme

Pour lancer l'assemblage et la simulation, utilisez ces commandes dans le terminal :

- Sous Linux/MacOs : ./simulateur exemple.txt
- Sous Windows : simulateur.exe exemple.txt

IV. Notes importantes

Le point positif de ce compilateur est principalement sa faible sensibilité à la casse. Contrairement à des langages stricts comme le C, où une simple erreur de majuscule empêche le programme de tourner, notre assembleur a été conçu pour être tolérant. Concrètement, le système interprète de manière identique les instructions, qu'elles soient écrites en majuscules (ADD), en minuscules (add) ou même mélangées. C'est équivalent pour les registres qui sont identifiés de la même manière s'ils sont écrits r1 que R1.

De plus, un autre de ses points forts est que l'hexadécimal est traité de la même manière que la base 10 : si l'on rentre #0xA c'est équivalent à rentrer #10. La structure proposée est nécessaire en revanche pour rentrer des valeurs en hexadécimal. C'est aussi le cas pour les entrées sur le IN, cependant il est important de noter que pour les entrées manuscrites la base 8 est aussi accepté, on a donc que 010 et 10 ne sont pas équivalent. La syntaxe pour la base 16 dans le IN dois être de cette façon : 0xNombre.

La rigidité du code assembleur ne permet pas qu'un code ayant un espace entre un label et les deux points soit correcte. Cependant, notre programme ne le prend pas en compte. Il ne faut pas en abuser le compilateur en tolère un seulement.

Un point négatif, malheureusement, notre programme ne gère pas les nombres flottants. Par exemple 5/2 renverra 2 et non 2.5 et en entrée également, 1.5 sera traduit en 1 et ne gardera pas sa valeur originelle.

Enfin, il est important de noter que nous avons pris le parti de tronquer l'exécution a 50 000 opérations afin d'éviter les boucles infinis, évidemment ce nombre prit aléatoirement est modulable.

V. Interprétation des résultats

1. Cas de succès

Si le fichier est correct vous verrez affichez :

a) Analyse syntaxique

C'est la première lecture du document, le compilateur découpe les lignes avec le label, les opcodes, et argument. Il vérifie que les opcodes existent, que les arguments soient valides et qu'il n'y a pas d'erreur de format, comme erreur de parenthèse, etc...

b) Résolution des symboles

C’est la deuxième partie de la compilation, il transforme les étiquettes par des numéros correspondant, par exemple JMP boucle sera transformé par JMP #12 afin que la machine puisse mieux comprendre.

c) (Table des symboles) Elle n’est pas appelée par défaut dans le code, mais peut l’être en retirant les // sur la fonction afficher_prog dans le main.

Cette partie de code n’est pas nécessaire à son fonctionnement mais permet d’avoir une idée de la structure du programme assembleur pour faciliter le débogage. Il affiche la liste de toutes les étiquettes, instructions et arguments trouvés dans votre code et leur adresse.

d) Code assemble

C’est la dernière étape de la compilation, il réécrit tout le code, mais "propre" : sans commentaires, tout en hexadécimal, et les valeurs formatées uniformément. Prêt a être convertie en binaire pour être interprété par l’ordinateur. Ecrit dans un fichier hexa.txt

e) L’exécution

C’est la dernière étape du processus, elle lance l’exécution du code écrit dans hexa.

2. Cas d’échec (Erreur)

Le processus s’arrête directement si une erreur est détectée, il y a 4 types d’erreur possibles, toutes les erreurs renvoient la ligne où le programme plante :

a) Erreur de syntaxe : Une instruction mal écrite ou inconnue, le programme affiche la ligne du programme où le code est mal écrit
b) Erreur de format : Oubli de parenthèse ou de #, le programme ne comprend pas ce qui est demandé et donc il s’arrête.
c) Erreur d’Overflow : Une valeur dépasse la capacité maximale, 16 bits.
d) Erreur de label : Un saut vers une étiquette qui n’existe pas.

Voici un exemple d’erreur :
-> Erreur de syntaxe ligne 5. ERREUR FORMAT ARG : '#40000' hors limites.

3. Warning (texte jaune), compilation effectuée aussi

Le programme compile mais signale des étrangetés, voici la liste des warnings possible :

a) Warning d'écriture : Il est inutile d'écrire ou de déplacer des valeurs dans le registre R0, car il est toujours nul. Donc cela renvoie un warning.
b) Warning de label : Si un label est défini mais jamais utilisé. La syntaxe est correcte mais le label est inutile, le compilateur le signale simplement.

VI. Tableau d'instruction autorisé

Mnémonique	**Code**	**Catégorie**	**Format attendu**	**Description / Action**
OR	0	Logique	OR Rd, Rn, Op2	Ou logique bit-à-bit (|)
AND	1	Logique	AND Rd, Rn, Op2	Et logique bit-à-bit (&)
XOR	2	Logique	XOR Rd, Rn, Op2	Ou exclusif bit-à-bit (^)
ADD	3	Arithmétique	ADD Rd, Rn, Op2	Addition (+)
SUB	4	Arithmétique	SUB Rd, Rn, Op2	Soustraction (-)
MUL	5	Arithmétique	MUL Rd, Rn, Op2	Multiplication (*)
DIV	6	Arithmétique	DIV Rd, Rn, Op2	Division entière (/)
SHR	7	Arithmétique	SHR Rd, Rn, Op2	Décalage de bits (Positif = Droite, Négatif = Gauche)
LDB	12	Mémoire	LDB Rd, (Rn+Imm)	Charge un octet (8 bits) depuis la RAM vers Rd
LDH	13	Mémoire	LDH Rd, (Rn+Imm)	Charge un demi-mot (16 bits) depuis la RAM vers Rd
LDW	14	Mémoire	LDW Rd, (Rn+Imm)	Charge un mot complet (32 bits) depuis la RAM vers Rd
STB	15	Mémoire	STB (Rn+Imm), Rs	Écrit l'octet de poids faible de Rs en RAM
STH	16	Mémoire	STH (Rn+Imm), Rs	Écrit les 2 octets de poids faible de Rs en RAM
STW	17	Mémoire	STW (Rn+Imm), Rs	Écrit le mot complet (32 bits) de Rs en RAM
JMP	21	Saut	JMP Label	Saut inconditionnel à l'adresse indiquée
JZS	22	Saut	JZS Label	Saut si Zéro Set (Dernier résultat == 0)
JZC	23	Saut	JZC Label	Saut si Zéro Clear (Dernier résultat != 0)
JCS	24	Saut	JCS Label	Saut si Carry Set (Retenue ou dépassement détecté)
JCC	25	Saut	JCC Label	Saut si Carry Clear (Pas de retenue)
JNS	26	Saut	JNS Label	Saut si Negative Set (Dernier résultat < 0)
JNC	27	Saut	JNC Label	Saut si Negative Clear (Dernier résultat >= 0)
IN	28	Système	IN Rd	Attend une saisie utilisateur et la stocke dans Rd

OUT	29	Système	OUT Rs	Affiche la valeur contenue dans Rs sur la console
RND	30	Système	RND Rd, Rs	Génère un nombre aléatoire entre 0 et Rs dans Rd
HLT	31	Système	HLT	Arrêt immédiat du processeur (Fin du programme)

VII. Instruction créable à partir de celle-ci :

Commande voulue	**Code réel à écrire (Traduction)**	**Explication**
MOV R1, R2 (Copier R2 dans R1)	ADD R1, R2, #0	On ajoute 0 à R2, le résultat va dans R1.
NEG R1, R2 (R1 = -R2)	SUB R1, R0, R2	On fait le calcul 0 - R2.
NOT R1, R2 (Inverse tous les bits)	XOR R1, R2, #-1	Le XOR avec -1 (tous les bits à 1) inverse le nombre.
CMP R1, R2 (Comparer pour sauter)	SUB R0, R1, R2	On soustrait. On se fiche du résultat (mis dans R0 poubelle), seuls les Flags (Zero, Neg) nous intéressent pour le prochain JZS ou JNS.

Push :

SUB R31, R31, #4 ; On décrémente le pointeur de pile de 4 octets

STW (R31), R1 ; On sauvegarde R1 à la nouvelle adresse

Pop :

LDW R1, (R31) ; On lit la valeur pointée par la pile vers R1

ADD R31, R31, #4 ; On incrémente le pointeur (on "libère" la place)